

FrontierAtlas Intelligence Graph

Production architecture for bulk acquisition and fresh signal ingestion

The runnable repository uses deterministic fixtures for repeatable assessment runs. Each fixture record carries a source-shaped URL and the source adapters expose the same async contract needed by live providers. Live adapters must persist raw responses and only emit records that have passed source and schema validation.

1. Scale to 500k+ records

Partition work by source and cursor/range. A coordinator places idempotent fetch jobs on a queue; stateless async workers use per-host concurrency limits, connection pooling, and backpressure. Store raw pages in object storage keyed by URL hash plus fetch timestamp, then run extraction and resolution as independently scalable stages. Checkpoints contain source, cursor, checksum, and schema version, so a worker can resume without duplicates. A 500k run scales by adding workers and queue partitions rather than changing scraper code.

2. Exact 413 and 429 strategy

Before an LLM call, measure UTF-8 payload size and split text into 12,000-character chunks with 500 characters of overlap. Chunk outputs are merged by stable entity keys. A 413 is a permanent input error for that request: rechunk once and never retry the same oversized payload. A 429 is retrievable: honor Retry-After when supplied, otherwise use exponential backoff with bounded jitter (0.5s, 1s, 2s, 4s), a per-provider token bucket, and a circuit breaker. Exhausted Gemini requests fall through to Groq, then DeepSeek; failures are recorded with provider, attempt, status, and payload hash.

3. Distributed freshness and anti-bot controls

Normalize each canonical URL, hash it with SHA-256, and claim it in Redis with a short lease using SET NX. Persist last_seen_published_at, response checksum, and source cursor. A 24-hour watermark is evaluated in UTC after parsing absolute and relative dates; records without reliable dates go to a review queue rather than being silently labeled fresh. For Cloudflare/Datadome and JavaScript-heavy sources, use an approved browser worker with Playwright, respect robots.txt and terms, low per-host concurrency, caching, and source APIs/RSS where available. Do not attempt CAPTCHA circumvention; route blocked pages to a compliant fallback or human review.

4. Storage and relationships

PostgreSQL is the system of record because it provides transactions, constraints, JSONB for raw/source-specific fields, and practical upserts for entity versions. pgvector stores embeddings for semantic candidate retrieval; deterministic RapidFuzz matching remains the final canonicalization gate. Relationship tables connect startups, products, papers, jobs, news, founders, and sources. Object storage retains immutable HTML/JSON evidence, while Redis handles leases, rate limits, and freshness watermarks. Every row keeps source URL, collectedAt, parser version, and evidence checksum for auditability.

5. Data contracts and operational guarantees

All emitted records include `schemaVersion` and `recordType`. Pydantic validation rejects missing required fields, invalid enum values, malformed timestamps, and negative metrics. CSV is a convenience export; canonical persistence should use typed tables. Metrics include fetch success rate, freshness pass rate, duplicate rate, unresolved entity rate, provider fallback rate, and queue latency. Alerts fire on freshness lag, error-rate spikes, and provider circuit-open events.

6. Security and compliance boundaries

Secrets are injected through environment variables and never written to CSV or logs. Raw content is access-controlled and retention-limited. Source terms, robots directives, rate limits, and provider policies are configuration, not hard-coded bypass targets. The assessment fixture mode is clearly labeled so generated rows cannot be mistaken for verified live intelligence.